

Airflow

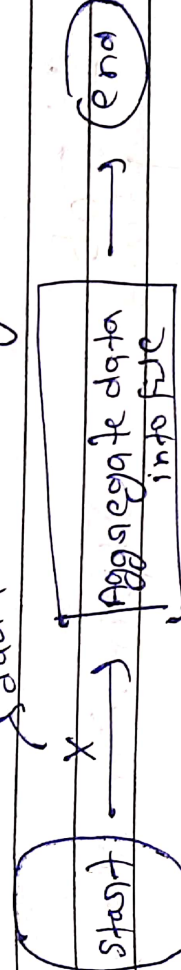
Before the Airflow error

→ Bash Script (Some kind of task is scheduling).

→ Cron Job

→ Python Code → Time.

↓
Loading



i) get access to file source.

ii) Load data to file.

iii) Aggregate data.

iv) Store data to sink.

→ these are the steps that we have follow.

→ We want to schedule the data loading in the hourly basis.

ex -

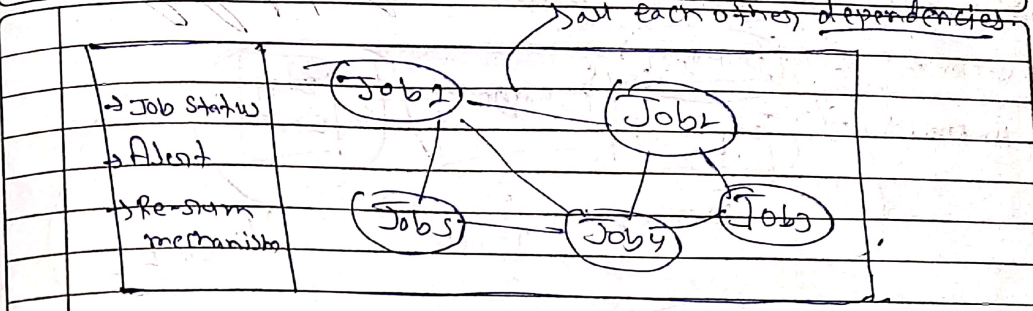
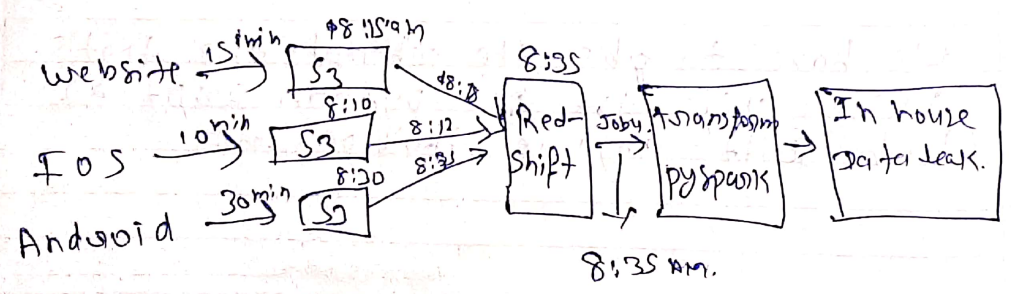
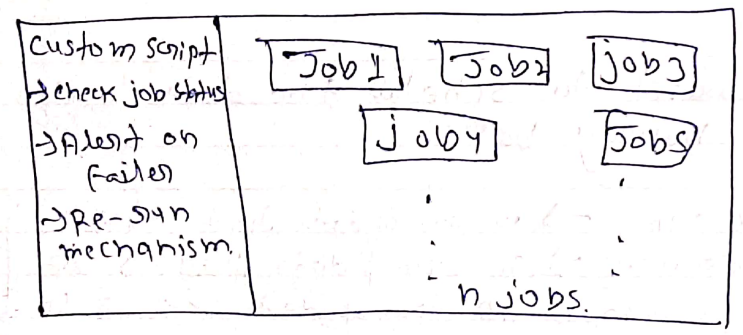
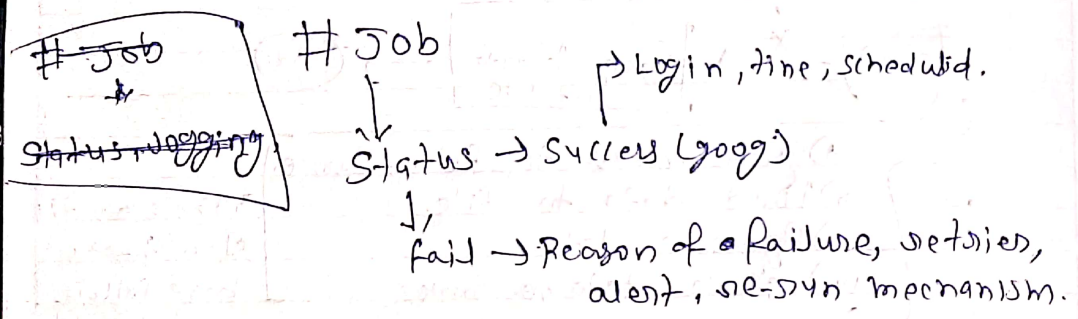
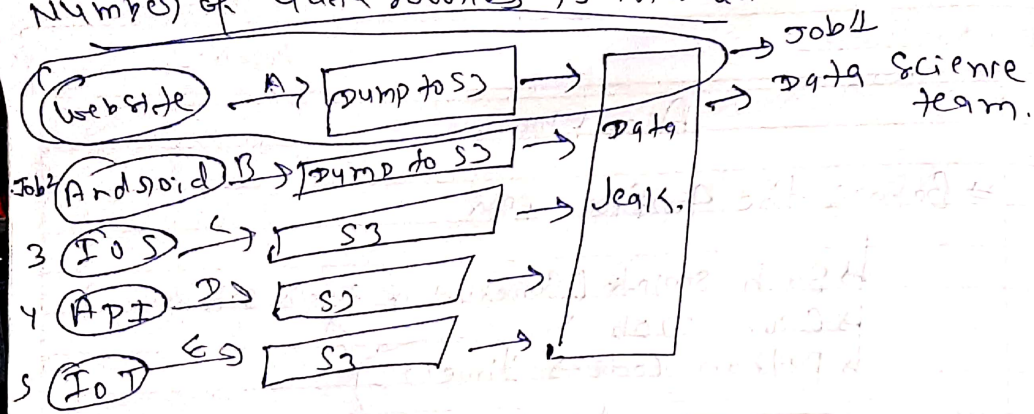
8.00 am → 10 rows of data load → 10

9.00 am → 20 rows of data load → 20

10.00 am → 30 rows of data load → 30

We have to able the schedule, the that's things using the python script or bash script anyone that perform.

Number of data sources is increased

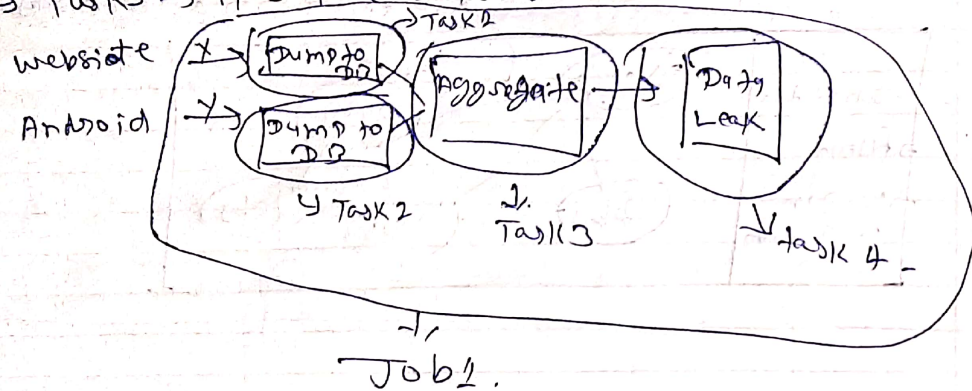


- * Services provided by Airflow -
 - monitoring: Job status, view, historic run get to know about duration.
 - manage failure: alert on failure, retries, timeout.
 - dependencies - check upstream data, syn jobs after jobs.
 - Backfills: re-run historic data ingestions.
 - Scalability: centralized system to manage all the jobs.
 - Deployment: version control orchestration

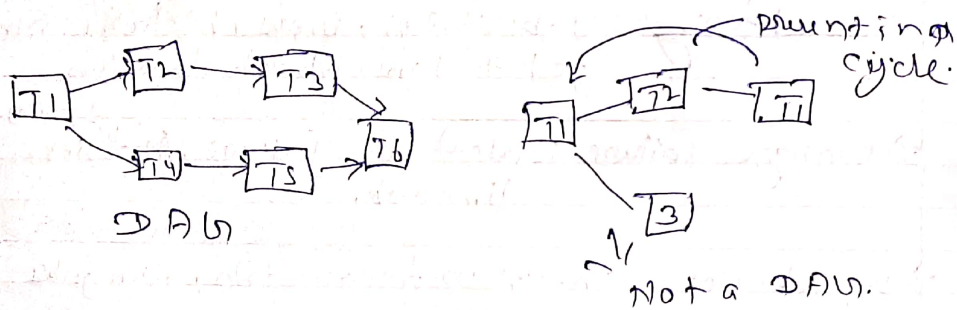
Teacher's Signature _____

* Airflow terminologies:-

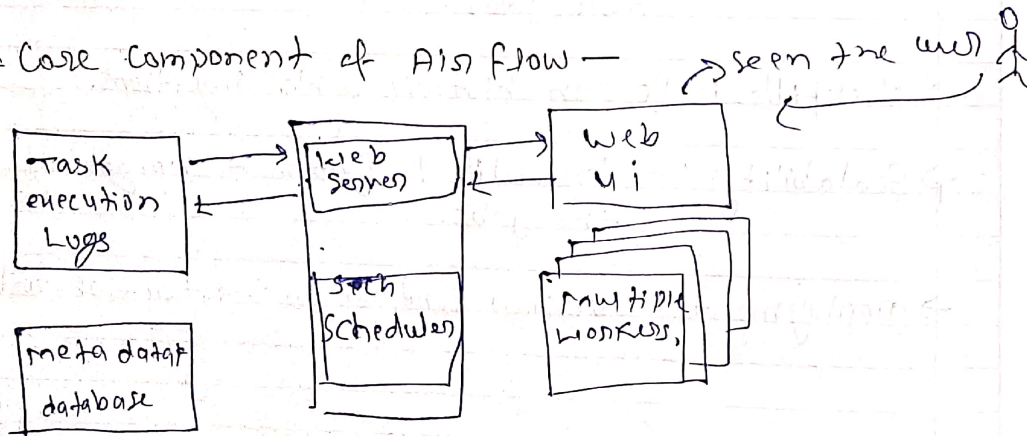
→ Tasks → it's the smallest unit of job.



DAG → (Directed Acyclic Graph)



Core Component of Airflow —



Expt. No. _____

Date _____
Page No. _____

→ Scheduler → Triggers scheduled workflows (Job) and submit the job to the executor.

Executor:- Handles to jobs/tasks.

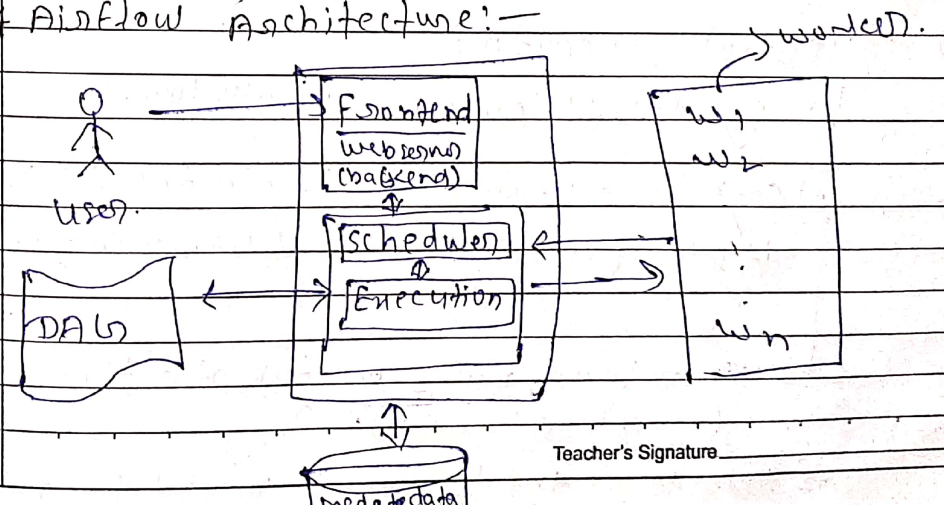
→ Workers → Worker actually runs/execute the task.

→ Web Server → UI to inspect, trigger, debug the DAG.

→ Metadata DB:- Stores info about DAG & task state.

→ DAG folder → directory where all a DAG's code will be stored and the scheduler will read the code from DAG folder itself.

* Airflow architecture:-



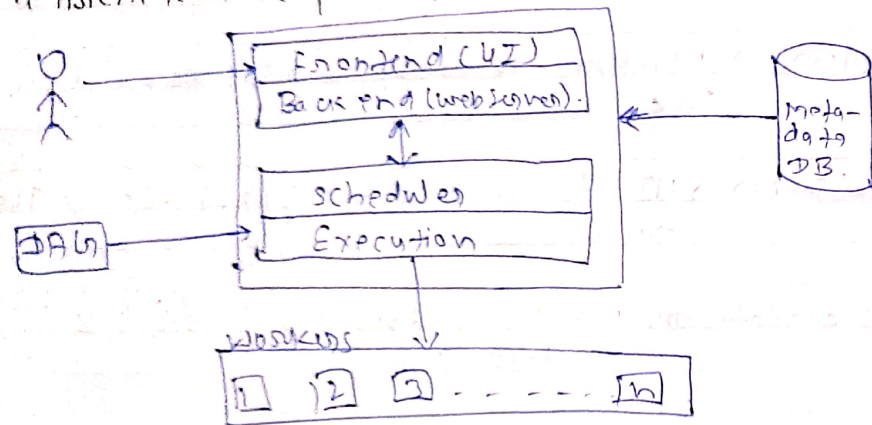
Teacher's Signature _____

↳ Airflow is an orchestration tool that helps in managing & scheduling jobs.

Job/task

Combination of task is job.

* Architecture of Airflow —



DAG is the directory where the python files having the job code will be stored.

* Type of executor:—

① Sequential Executor

- ↳ By default executor in airflow.
- ↳ Seqn executor runs within the scheduler itself. (Executor & scheduler will share the same machine).
- ↳ It can run only one task at a time. (no worker is present)

→ Sqlite is the by default metadata database that is used. (in-memory DB that

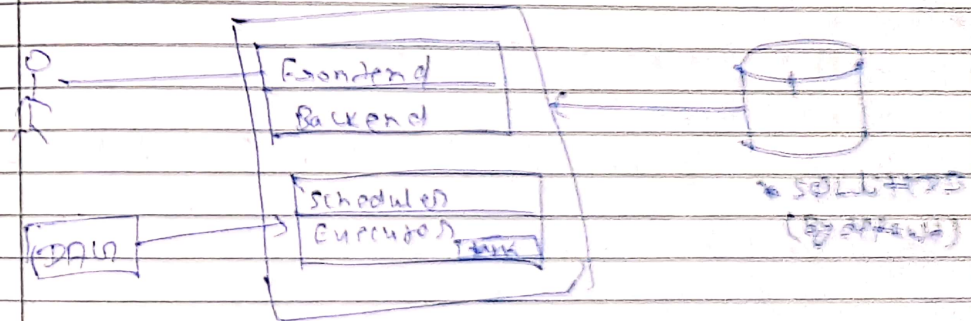
utilizes SQL Engine to store data.)

→ Pros:—

- No Setup is required (by default executor)
- Light weight
- Cheap
- Low complexity

→ Cons:—

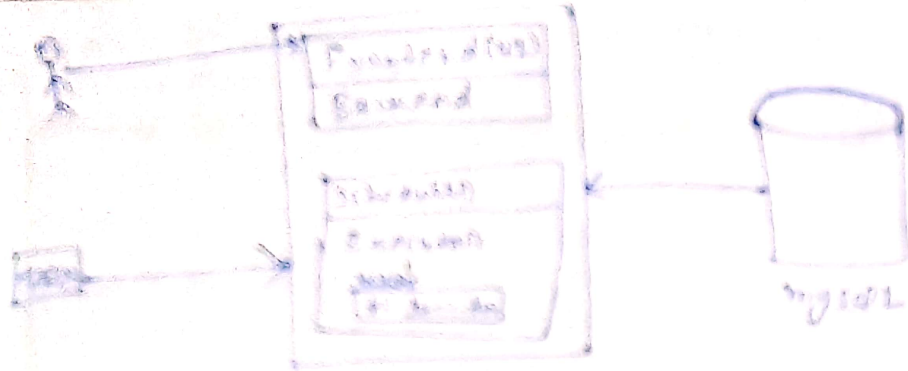
- Not production ready
- Not Scalable
- Slow (no workers)
- Single point of failure



② Local Executor —

- The executor runs within the scheduler itself.
- We can initiate multiple tasks within the executor.

Teacher's Signature _____



• Pros :-

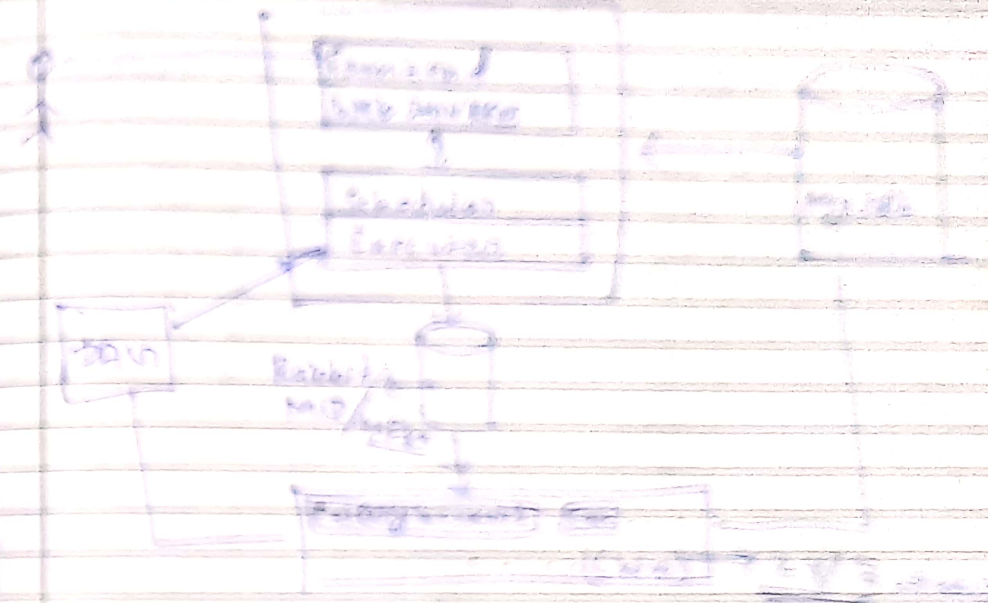
- multiple task within the execution (Faster than local execution)
- Easy to setup
- cheap (hardware cost, maintenance cost)
- light weighted (not utilized large amount of resource)

• Cons :-

- Not suitable for production
- No scalable
- Single point of failure (not fault tolerant)
- Cannot handle high load.

■ Celery Execution :-

- this execution is most widely used in the industry
- This execution runs on a dedicated machine (different machine than scheduler).
- it can handle multiple jobs/tasks at the same time.
- We will utilize the queuing service for the task execution.



• Pros :-

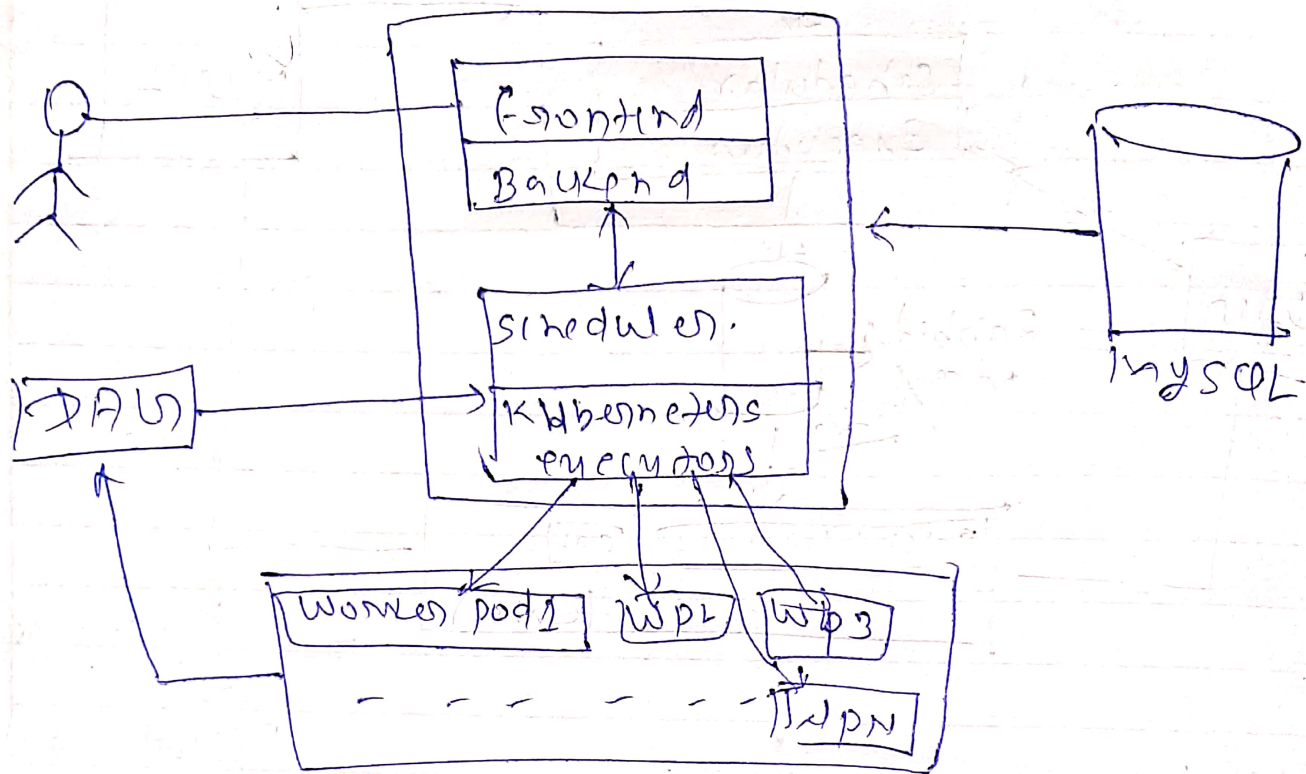
- production ready
- Highly Scalable (horizontal scaling easy)
- Fault tolerant (multiple workers can handle the task)
- Can work well with high load
- Concurrent execution is possible (Fast)

• Cons :-

- High maintenance
- It is more complex than sequential execution
- Queuing system is the bottleneck
- High volume of requests - dealing with them

Kuban Kubernetes Executor: —

- This executor is the latest.
- The workers run on a dedicated pods.
- Uses Kubernetes API to manage the pods.



• Pros: —

- production ready executor
- fault tolerant
- it can assign individual resource to each task
- Highly Scalable (it can scale up from zero as well as it can scale down to zero).
- Cost efficient as compared to celery executor

• Cons —

- Kubernetes expert will be required.
- cold start problem: (launching of pods for the task takes few second of time)